Canonical Even-Index Representations and Recursive Decomposition of Rectilinear Polygons

Wai-Shing Luk

***Abstract*—Rectilinear polygons are the native geometry of VLSI physical design, where coordinates are integers and a polygon has at most about one hundred vertices. We study the even-index representation, which stores every other boundary vertex and recovers the omitted vertices by an x-first rule. We give an exact structural predicate for canonical representations, a template-method construction and monotone-hull algorithm with a collinear-cleanup step, and a recursive decomposition into convex pieces and rectangles that distinguishes explicit from implicit 270-degree vertices. We prove that the x-first expansion is a closed rectilinear polygon if and only if every cyclic pair of stored points differs in both coordinates, that collinear cleanup preserves the represented boundary and area, and that recursive cutting preserves area and convexity. The hulls run in $O(m)$ and the decomposition in $O(n^2)$, appropriate for the small polygons of the domain. We validate the predicates and invariants in Python and C++20; the suites pass (627 Python tests; 139 C++ cases with 934 assertions) and expose a representation-specific caveat: area checks alone cannot replace canonical validation.**

I. Introduction

A modern integrated circuit is a physical object with billions of components, and physical design turns an abstract netlist into the polygons that a foundry can fabricate [1], [2], [3]. The geometry of the layout is overwhelmingly *rectilinear*: every edge is horizontal or vertical, and every coordinate is an integer. Rounding is not merely imprecise but illegal, because two edges that should be collinear may miss and a design-rule check fails. At the same time, the individual objects are small: a polygon has at most about one hundred vertices, a net a handful of pins, and a chip a score of metal layers. Quadratic algorithms are therefore acceptable when they are exact and simple.

This paper is about the *representation* of a rectilinear polygon and the algorithms that consume it. The usual representation is the full boundary, a cyclic list of all n vertices. Because consecutive edges alternate between horizontal and vertical, every other vertex determines the omitted one under a suitable convention. The resulting *even-index* representation stores only $m = n/2$ points. It is compact, but it is also fragile: the expansion convention must be fixed, and a stored list can represent a degenerate or non-canonical boundary if consecutive points share an axis or coincide. Area checks do not detect such defects.

We make the following contributions.

1. We define the x-first even-index encoding and its expansion, and prove that the expansion is a closed rectilinear polygon if and only if every cyclic consecutive pair of stored points differs in both coordinates (Section III).
2. We give an exact structural predicate for canonical representations, analyze its complexity, and show why geometric area checks are insufficient (Section IV).
3. We present a template-method treatment of creation and monotone hulls, prove the correctness of a collinear-cleanup step for even-index hulls, and compose the passes into a rectilinear convex hull (Section V).
4. We describe a recursive decomposition into convex pieces and rectangles, distinguishing explicit from implicit 270° vertices, and prove the area, convexity, and canonicality invariants (Section VI).
5. We validate the predicates and invariants in Python and C++20, report the test results, and discuss the limitations of the representation-specific canonical predicate (Section VII, Section VIII).

The remainder of the paper is organized as follows. Section II reviews related work. Section III fixes notation and proves the representation lemmas. Section IV develops the canonical predicate. Section V presents monotone hulls and collinear cleanup. Section VI develops recursive decomposition. Section VII reports the implementation and tests. Section VIII discusses scope and threats to validity, and Section IX concludes.

II. Related Work

Rectilinear polygons are a classical topic in computational geometry and a workhorse of VLSI layout. The standard textbook treatment covers monotone polygons, convex decomposition, and rectilinear convex hulls [1], [2]. The VLSI literature uses the same primitives for layout fracturing and for decomposing L-shaped obstacles into rectangles; double-patterning and resolution-enhancement flows make the rectangle decomposition a manufacturing requirement rather than a convenience [4], [5].

The even-index representation studied here is a compact encoding rather than a new geometric object. Its interest is algorithmic: it halves the storage, makes translation an $O(1)$ update of the origin, and turns the canonicality question into a local structural condition. The condition is deliberately stricter than geometric simplicity: it enforces the x-first convention, not only the absence of self-intersections. This distinction is central to the verification methodology of Section VII.

Exact integer geometry avoids the degeneracies that arise from floating-point predicates [6]. Our algorithms use only comparisons and integer arithmetic; the area and orientation tests are exact. The decomposition is a recursive cut at concave vertices, in the spirit of the classical convex-decomposition algorithms, with two additional concerns: the even-index representation introduces *implicit* 270° vertices, and the output must be canonical whenever the algorithm promises it.

III. Preliminaries and Representation

A. Rectilinear Polygons

Let $V = (v_0, \ldots, v_{n-1})$ be a cyclic sequence of points in $\mathbb{Z}^2$. It is a *rectilinear polygon* if consecutive edges are axis-parallel and alternate between horizontal and vertical. We call V the *full boundary*. The signed area is

$$A(V) = \frac{1}{2} \sum_{i=0}^{n-1} (x_i y_{i+1} - x_{i+1} y_i), \tag{1}$$

where indices are modulo n; $A > 0$ means anticlockwise and $A < 0$ clockwise.

B. Even-Index Encoding

Assume n is even. The *even-index encoding* of V is

$$S = (s_0, \ldots, s_{m-1}) = (v_0, v_2, \ldots, v_{n-2}), \qquad m = n/2. \tag{2}$$

The omitted odd-index vertices are implicit. The encoding is useful only if the expansion rule is fixed.

Definition 1 (x-first expansion). For consecutive stored points $s_i = (x_0, y_0)$ and $s_{i+1} = (x_1, y_1)$, insert the implicit vertex

$$I_i = (x_{i+1}, y_i) \tag{3}$$

before s_{i+1}, including the closing pair $s_{m-1} \to s_0$. The resulting cyclic sequence is $E(S)$.

The diagonal chord $s_i s_{i+1}$ is not a boundary edge; the boundary follows the horizontal edge to I_i and then the vertical edge to s_{i+1}. The y-first form (x_0, y_1) reconstructs a different polygon and is not used.

Lemma 1. $E(S)$ is a closed rectilinear polygon if and only if every cyclic consecutive pair of stored points differs in both coordinates.

Proof. Each pair contributes a horizontal edge followed by a vertical edge. If $x_i = x_{i+1}$, the horizontal edge has zero length; if $y_i = y_{i+1}$, the vertical edge has zero length. Either case produces a repeated vertex and a degenerate boundary. Conversely, if both coordinates differ for every pair, both inserted edges have positive length and the expansion alternates horizontal and vertical, so it is a closed rectilinear polygon. □

Lemma 2. The signed area is invariant under the x-first expansion: $A(E(S)) = A(V)$ whenever $V = E(S)$.

Proof. Each inserted vertex lies on an axis-aligned edge between two consecutive vertices of the original boundary; splitting an edge at a collinear point contributes zero net area. Summing over all insertions gives the claim. □

Algorithm 1 IsCanonical(S)

```
Require: stored points S = (s_0, ..., s_{m-1})
if m < 2 then
    return True
end if
for i ← 0 to m − 1 do
    for j ← i + 1 to m − 1 do
        if s_i = s_j then
            return False
        end if
    end for
end for
for i ← 0 to m − 1 do
    j ← (i + 1) mod m
    if x_i = x_j or y_i = y_j then
        return False
    end if
end for
return True
```

C. Minimal Encodings

Two stored points represent a rectangle: $E(S)$ has four corners. Three stored points represent a monotone, L-shaped piece: $E(S)$ has six corners. These are not degenerate cases; they are the base cases of the decomposition. A stored list with two points that share an axis is *not* a rectangle in this encoding: its expansion collapses one edge, by Lemma 1.

IV. Canonical Representation

Definition 2 (canonical). A stored sequence $S = (s_0, \ldots, s_{m-1})$ is canonical if

1. the points s_i are pairwise distinct, and
2. for every i (indices modulo m),

$$x_i \neq x_{i+1} \quad \text{and} \quad y_i \neq y_{i+1}. \tag{4}$$

The predicate is structural: it constrains the representation, not the geometry. Algorithm 1 checks it directly.

The duplicate scan is $O(m^2)$ in the worst case; sorting the points reduces it to $O(m \log m)$. The consecutive-pair scan is $O(m)$. For the small m of the domain, the simple quadratic scan is preferable.

The predicate is not equivalent to an area check. Consider $S = ((2,1),(2,0),(3,-3))$. Its expansion is

$$E(S) = (2,1),(2,0),(3,0),(3,-3),(2,-3),$$

which contains the consecutive vertical segments $(2,1) \to (2,0)$ and $(2,-3) \to (2,1)$; the stored list is not canonical. Its canonical form is $((2,1),(3,-3))$, a rectangle. The signed area of the two representations is the same, so an area check would accept the defective one. Table I lists the cases used by the test suite.

A geometric caveat is worth stating explicitly. The predicate enforces the x-first even-index convention. A geometrically valid rectangle can be written with a same-axis pair

Table I: Canonical predicate test cases.

Stored points	Canonical	Reason
$(2,1),(2,0)$	no	same x
$(2,1),(3,-3)$	yes	rectangle corners
$(2,1),(2,0),(3,-3)$	no	zero change in x
$(0,0),(0,0)$	no	duplicate
$(0,0),(0,1),(0,2)$	no	vertical run
$(0,0),(1,0),(2,0)$	no	horizontal run
$(0,0),(1,1)$	yes	rectangle

and still fail the predicate; the remedy is to rewrite the representation, not to change the geometry. This is why the predicate is checked after hull construction and before decomposition, and why a failed predicate is a representation error rather than a numerical error.

A. Non-Canonical Examples

The creation routines return an ordering, not necessarily a canonical list. Fig. 1 shows the two orderings produced by `create_xmono` and `create_ymono` on the hand-written twelve-point fixture used by the test suite. Both are valid even-index sequences, but neither is canonical. In the x-monotone ordering the violating triples are $(-3,-4) \to (0,-4) \to (3,-4)$ and $(-2,-1) \to (-2,4) \to (-2,2)$; in the y-monotone ordering they are $(-3,-4) \to (0,-4) \to (3,-4)$, $(-2,2) \to (2,2) \to (-2,2)$, and $(-2,4) \to (1,4) \to (-4,4)$. Each triple contains two consecutive collinear segments. Applying the matching hull and the collinear cleanup of Section V removes the defects and yields a canonical list.

V. Monotone Hulls

A. Monotonicity

A rectilinear polygon is *x-monotone* if every vertical line meets it in a single interval, and *y-monotone* if every horizontal line does. The following characterization is classical and is the basis of the two-pass hull.

Lemma 3. A rectilinear polygon is rectilinearly convex if and only if it is both x-monotone and y-monotone.

Proof. If the polygon is convex, a line in either direction meets it in a single interval. Conversely, if the polygon is not convex, a line in one of the two directions meets it in two disjoint intervals; the corresponding monotonicity fails. □

Fig. 2 shows the x-monotone stage and Fig. 3 the resulting rectilinear convex hull of a point set.

B. Monotone Hull Algorithm

The input is a stored even-index sequence S and an orientation flag $\sigma \in \{\text{CLOCKWISE}, \text{ANTICLOCKWISE}\}$. The algorithm finds the minimum and maximum vertices in the sweep direction, builds a circular doubly linked list, and walks the two chains from minimum to maximum and back. At a vertex with previous, current, and next points p, c, n, it computes

$$\Delta = (y_c - y_p)(x_n - x_c). \tag{5}$$

Algorithm 2 MonotoneHull(S, σ, dir)

Require: stored points S, orientation σ, sweep direction dir
Ensure: a monotone hull in the even-index representation
$v_{\min} \leftarrow \arg\min_{v \in S} dir(v)$; $v_{\max} \leftarrow \arg\max_{v \in S} dir(v)$
build a circular linked list over S
for each chain $v_{\min} \to v_{\max}$ and $v_{\max} \to v_{\min}$ **do**
  $c \leftarrow$ first vertex of the chain
  while $c \neq$ stop vertex **do**
    $p \leftarrow prev(c)$; $n \leftarrow next(c)$
    if $dir(c) > dir(n)$ **or** $dir(p) > dir(c)$ **then**
      $\Delta \leftarrow (y_c - y_p)(x_n - x_c)$
      if Δ has the wrong sign for σ **then**
        detach c; $c \leftarrow p$
      else
        $c \leftarrow n$
      end if
    else
      $c \leftarrow n$
    end if
  end while
end for
return CleanupCollinear(S)

A vertex is removed when the sweep coordinate violates monotonicity and Δ has the wrong sign for the orientation. Removal is $O(1)$ in the linked list. Algorithm 2 states the procedure.

Proposition 1. Algorithm 2 terminates and returns a monotone hull. Its running time is $O(m)$.

Proof. Every detach strictly decreases the number of vertices, so the loop terminates. A vertex is detached only when it violates monotonicity or turns in the wrong direction; the surviving vertices form a monotone chain. Each vertex is visited $O(1)$ times, and each detach is $O(1)$. □

C. Collinear Cleanup

A monotone hull can still contain *collinear* even-index points. In the x-first encoding, two consecutive points with the same y make the second point lie in the interior of a horizontal edge; two with the same x make the first point lie in the interior of a vertical edge. Algorithm 3 removes such points until no adjacent pair shares an axis.

Lemma 4. Algorithm 3 preserves the represented boundary and the signed area.

Proof. A removed point lies in the interior of a straight boundary edge. By Lemma 1, deleting it merges two collinear edges into a single edge; the polygon is unchanged as a point set. The area is therefore unchanged. □

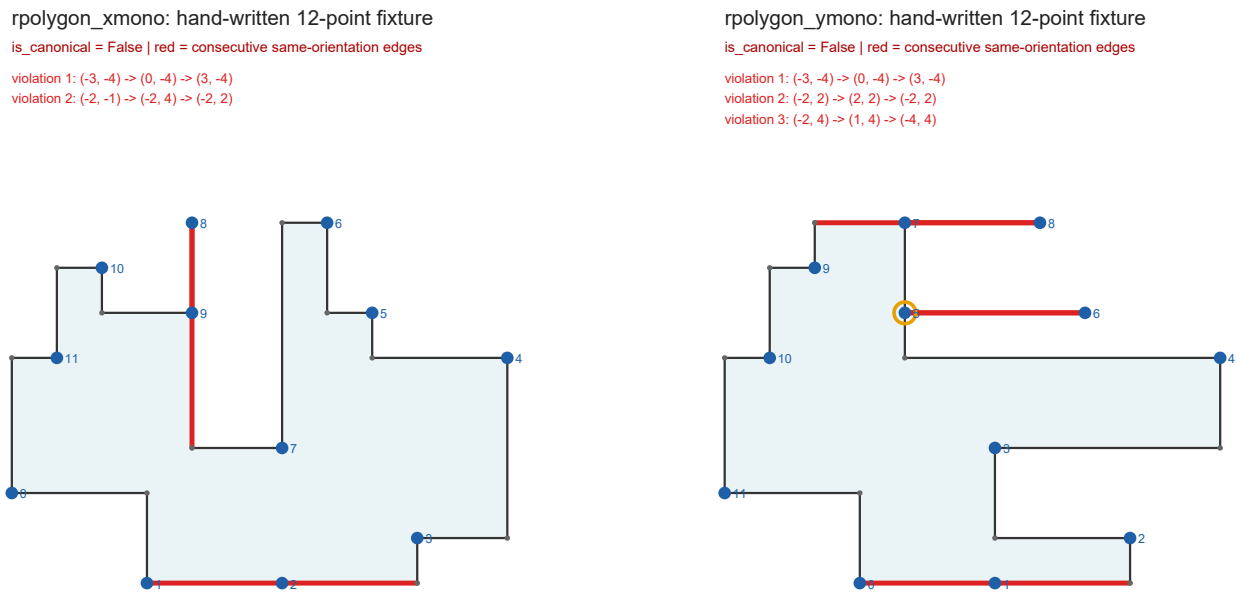


Figure 1: Non-canonical outputs of the creation routines on the hand-written twelve-point fixture: (left) `create_xmono`; (right) `create_ymono`. Red segments are consecutive collinear edges; the annotations in the figures list the violation triples. Both lists are valid even-index sequences but fail IsCanonical.

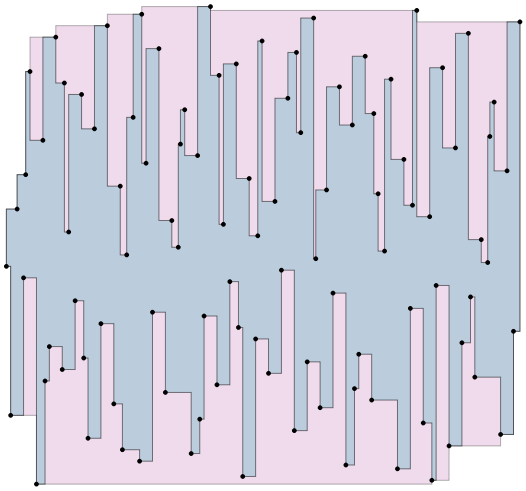


Figure 2: The x-monotone hull stage, before the y-monotone pass completes the rectilinear convex hull.

D. Composition

Applying the x-monotone hull and then the y-monotone hull yields the rectilinear convex hull. Each pass is $O(m)$, so the composition is $O(m)$, and both passes are idempotent on canonical inputs. Fig. 3 shows the result. The creation routines of Section III produce an ordering, not necessarily a canonical representation; the hull and cleanup are what make the output canonical.

Algorithm 3 CleanupCollinear(S)

Require: stored points S
Ensure: a canonical representative of the same boundary

```
changed ← True
while changed and |S| > 2 do
    changed ← False
    for i ← 0 to |S| − 1 do
        j ← (i + 1) mod |S|
        if y_i = y_j then
            remove s_j; changed ← True; break
        else if x_i = x_j then
            remove s_i; changed ← True; break
        end if
    end for
end while
return S
```

VI. Recursive Decomposition

A. Three Steps

The decomposition follows three steps.

1. **Convex cut:** remove concavities and produce convex pieces.
2. **Explicit 270°:** resolve reflex corners that are stored vertices.
3. **Implicit 270°:** resolve reflex corners introduced by the x-first expansion.

The final pieces are rectangles, suitable for layout fracturing and design-rule checking [4], [5]. We focus on the convex cut, which is the shared recursive skeleton; the explicit and implicit variants differ in the concave-vertex predicate and in the base cases.

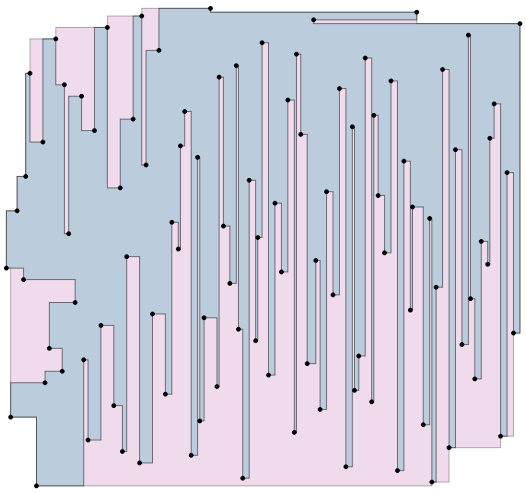


Figure 3: The rectilinear convex hull of a point set, obtained by the x- then y-monotone passes.

Algorithm 4 CUT(v_1)

Require: a circular list of stored points
Ensure: a list of convex pieces
$v_2 \leftarrow next(v_1)$; $v_3 \leftarrow next(v_2)$
if $v_3 = v_1$ **then**
  return $[v_1, v_2]$ ▷ 2-point rectangle
end if
if TRIANGLEBASE **and** $next(v_3) = v_1$ **then**
  return $[v_1, v_2, v_3]$ ▷ 3-point monotone piece
end if
$c \leftarrow$ FINDCONCAVE(v_1)
if $c =$ NIL **then**
  return all vertices ▷ already convex
end if
$(v_{\min}, vertical) \leftarrow$ FINDMINDIST(c)
$v_{\text{new}} \leftarrow$ INSERTCUT($c, v_{\min}, vertical$)
return CUT(c) ∪ CUT(v_{new})

B. Cut Primitive

At each step the algorithm finds a concave vertex, selects the nearest vertex along an axis-parallel line, inserts a cut point, and splits the circular list into two polygons. The cut primitive is stated in Algorithm 4.

A vertex is concave when the two incident edges turn in opposite coordinate senses and the area test has the correct sign. The nearest visible vertex is found by a Manhattan-distance scan; a horizontal or vertical cut is inserted at the intersection with the target edge. The cut adds one vertex and splits the cycle, so the recursion terminates.

C. Base Cases and Complexity

Two stored points form a rectangle and three form a monotone piece; the convex variant stops at these cases. The explicit variant has no three-point base case and continues until no concave vertex remains. If a cut splits a polygon of size n into pieces of size k and $n - k + 1$, the recurrence is

$$T(n) = T(k) + T(n - k + 1) + O(n), \tag{6}$$

and the worst case is $O(n^2)$. The space is $O(n)$ apart from the output.

D. Explicit and Implicit 270° Vertices

The compressed U-shape contains both kinds of reflex corner. A stored vertex such as $(2, 1)$ is an *explicit* 270° vertex; the implicit vertex $(1, 1)$ created by the x-first expansion is an *implicit* 270° vertex. The convex cut handles the shape, while the explicit and implicit cuts handle the corner types. Table II summarizes the variants.

E. Correctness Invariants

Proposition 2 (area). The sum of the signed areas of the pieces equals the signed area of the input.

Proof. Each cut inserts a segment whose two sides are traversed in opposite directions; the signed contributions cancel. Induction over the recursion gives the claim. □

Proposition 3 (convexity). Every returned piece is rectilinearly convex.

Proof. A piece is returned either at a base case (a rectangle or a monotone piece, both convex) or when the concave-vertex search returns nil, which by construction means that no reflex corner remains. □

Proposition 4 (canonicality). Every returned piece is a valid even-index list. When the variant guarantees it, the piece also satisfies ISCANONICAL.

Proof. The cut inserts a point at an axis-parallel intersection, so the stored sequence remains a valid even-index sequence. Canonicality is a stronger condition; the rectangle pieces satisfy it by construction, and the hull and cleanup phases establish it for the pieces that require it. □

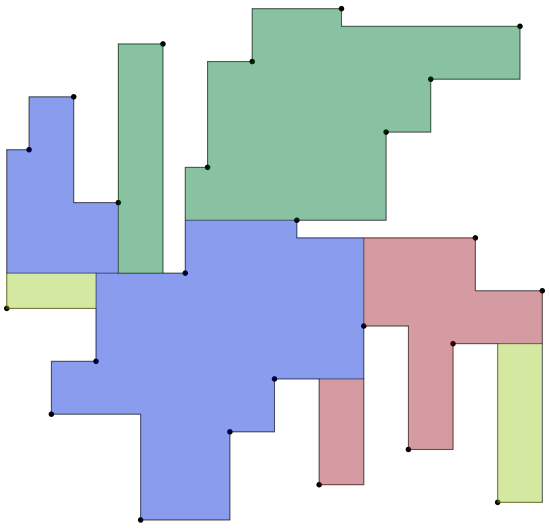


Figure 4: Convex decomposition of a concave rectilinear polygon into non-overlapping convex pieces (colored); vertices are marked.

The rectangle cut is illustrated in Figure 5 on the twelve-point fixture. The seven rectangles have areas 6, 2, 1, 2, 4, 4, and 1, which sum to the original area 20.

Table II: Cut variants and their guarantees.

Variant	Concave test	Base case	Output
Convex	reversal + area sign	2 or 3 points	convex pieces
Explicit	reversal + area sign	2 points	convex pieces
Implicit	implicit corner test	2 points	convex pieces
Rectangle	–	2 points	rectangles

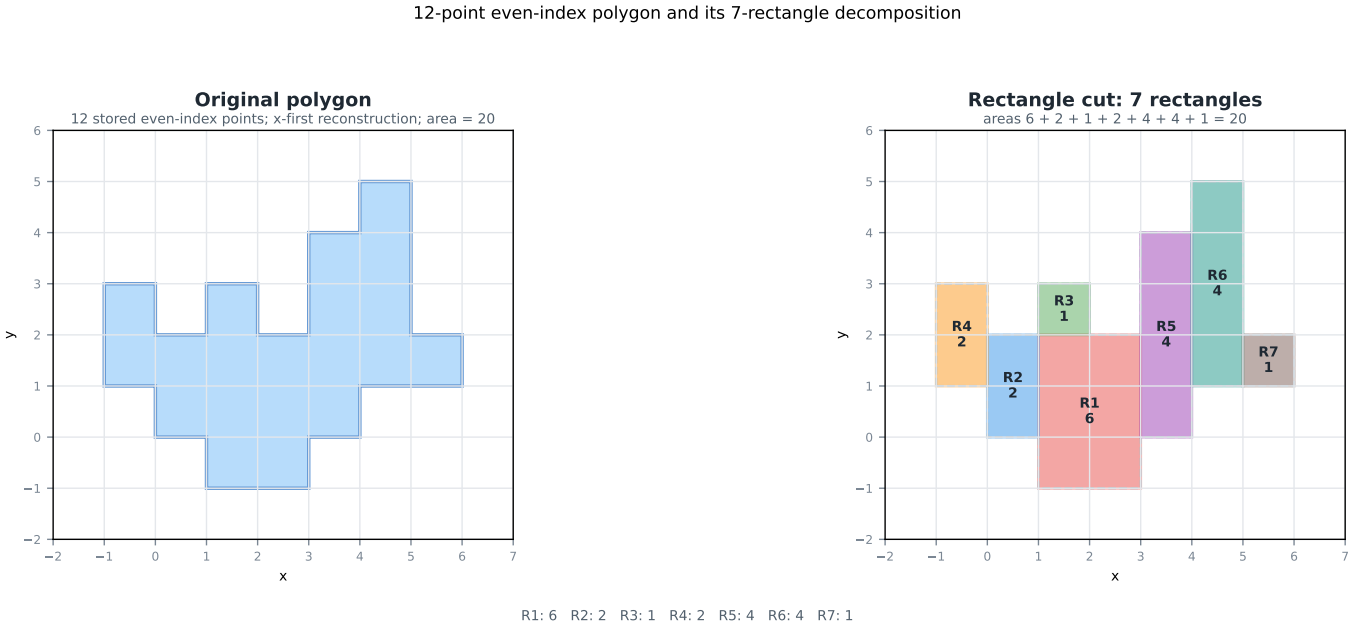


Figure 5: The twelve-point even-index polygon (left) and its decomposition into seven rectangles (right). The rectangle areas sum to the original area 20.

VII. Implementation and Experiments

A. Implementation

The algorithms are implemented in Python and C++20. The Python package `physdes` provides the reference implementation; the C++ project `physdes-cpp` provides a header-and-source port built with xmake. Both implementations expose the same predicates: IsCanonical, the monotone hulls, the convex hull, and the cut variants. The representation invariant is checked after hull construction and before decomposition, so a representation error is reported at the point where it is introduced rather than as a downstream geometric failure.

B. Test Design

The tests cover four layers.

1. **Predicate tests.** The cases of Table I, plus duplicate points and same-axis runs.
2. **Hull tests.** Create an ordering, apply the matching monotone hull, and assert IsCanonical on the result. The collinear-cleanup step is exercised by the twelve-point fixture whose x-hull is not canonical before cleanup.
3. **Cut tests.** Assert convexity, signed-area preservation, and canonicality on the pieces where the variant guarantees it.
4. **Cross-language tests.** The same fixtures are run in Python and C++.

C. Results

Table III reports the suite sizes. The Python suite passes 627 tests. The C++ suite passes 139 test cases with 934 assertions under xmake. The canonical predicate, the hull-before-check assertions, and the area checks on the cut pieces all pass. The tests also expose a limitation: three mixed even-index cut fixtures do not assert canonicality, because their pieces can still contain implicit collinear points. The cut algorithm is correct for area and convexity, but canonical output for every variant requires the cleanup pass. We do not report wall-clock timings; the contribution is exactness and canonicality, not a performance study.

Figure 6 shows the twelve-point fixture under the current convex cut and under the validated re-cut. The current cut returns a piece that is not convex; the re-cut splits it into three convex pieces with areas 10, 9, and 1, summing to 20.

VIII. Discussion and Limitations

Representation-specific predicate. IsCanonical enforces the x-first even-index convention. A geometrically valid rectangle can be written with a same-axis pair and still fail the predicate. The remedy is to rewrite the

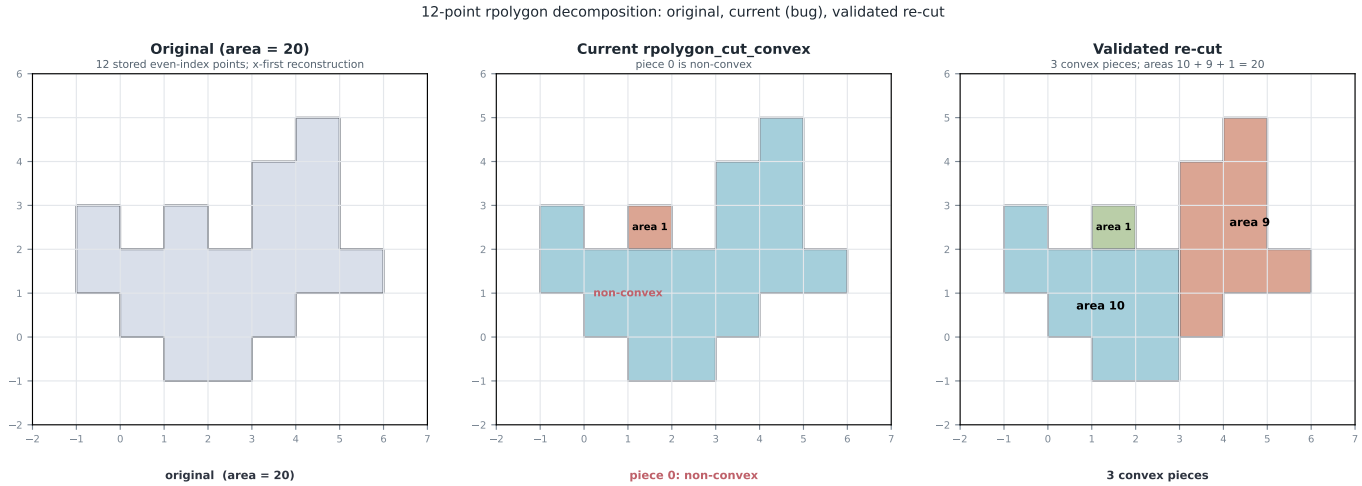


Figure 6: The twelve-point fixture: original polygon (left), current convex cut with a non-convex piece (middle), and validated re-cut into three convex pieces with areas 10, 9, and 1 (right).

Table III: Verification results.

Suite	Cases / assertions	Result
Python predicate and hull tests	627 tests	pass
C++ `test_recti`	139 cases, 934 assertions	pass
Canonical predicate	Table I	pass
Area preservation	all cut tests	pass

representation, not to change the polygon. This is a feature of the encoding, but it means that a canonicality failure is not, by itself, a geometric error.

Synthetic inputs. The evaluation uses generated point sets and hand-built fixtures, not foundry data. The claims are algorithmic: exactness, termination, area preservation, and canonicality. They are not signoff claims.

Quadratic scaling. The decomposition is $O(n^2)$. For $n \leq 100$ this is appropriate; for $n > 10^3$ a sub-quadratic method is an open problem.

Cut completeness. The convex and explicit cuts preserve area and convexity, but some pieces still contain implicit collinear points. The cleanup pass closes that gap; a cut algorithm that emits canonical pieces directly is future work.

Robustness. Integer arithmetic makes the predicates exact. Degenerate input handling beyond the stated invariants is outside the present scope; exact geometric predicates in the presence of degeneracies are a separate subject [6].

IX. Conclusion

We studied the even-index representation of rectilinear polygons and the algorithms that create, hull, and decompose them. The x-first expansion is characterized exactly by a local condition on consecutive stored points; the canonical predicate turns that condition into an exact structural test; the monotone hull and collinear cleanup produce canonical convex hulls; and the recursive cut produces convex pieces and rectangles while preserving area and convexity. The implementation is shared across Python and C++20 and is validated by predicate, hull, and cut tests. Open problems include sub-quadratic decomposition, canonical output for every cut variant without a post-processing pass, integration with detailed routing and design-rule checking, and three-dimensional rectilinear decomposition.

X. Code Availability

The Python implementation is in `physdes-py` and the C++20 implementation in `physdes-cpp`, both under the `luk036` organization. The Python tests run with `pytest`; the C++ tests build and run with `xmake build test_recti` and `xmake run test_recti`.

[1] N. A. Sherwani, *Algorithms for VLSI physical design automation*, 3rd ed. Boston, MA: Kluwer Academic Publishers, 1999.

[2] A. B. Kahng, J. Lienig, I. L. Markov, and J. Hu, *VLSI physical design: From graph partitioning to timing closure*. Dordrecht: Springer, 2011.

[3] S. M. Sait and H. Youssef, *VLSI physical design automation: Theory and practice*. Singapore: World Scientific, 1999.

[4] A. B. Kahng, C.-H. Park, X. Xu, and H. Yao, "Layout decomposition for double patterning lithography," in *Proc. IEEE/ACM int. Conf. Computer-aided design (ICCAD)*, 2008, pp. 465–472. doi: 10.1109/ICCAD.2008.4681616.

[5] L. W. Liebmann, "Layout impact of resolution enhancement techniques: Impediment or opportunity?" in *Proc. Int. Symp. Physical design (ISPD)*, 2003, pp. 110–117. doi: 10.1145/640000.640026.

[6] H. Edelsbrunner and E. P. Mücke, "Simulation of simplicity: A technique to cope with degenerate cases in geometric algorithms," *ACM Trans. Graph.*, vol. 9, no. 1, pp. 66–104, 1990.